SECURITY

Security Architecture & Threat Model

Revision: 2 **Last modified:** 2026-09-22T00:00:00Z

> **ARCHITECTURE AMENDMENT — 2026-09-22 (HXC-159 Phase B3).** This threat model covers the FUTURE, ASPIRATIONAL nano-task engine + its (never-built) "SuperBridge MCP" server (§3.5/§5) — independently re-derived twice (GitHub `total_count: 0`; npm exact-name lookup `HTTP 404`) not to exist as software. This project's REAL Bridge architecture is 6 layers with no MCP execution tier; see `../README.md#architecture-amendment-2026-09-22`. §4.3 has ALSO been genericised in this same pass — it previously carried an un-genericized cross-project ("ATMOSphere project mandate") reference that did not belong in this project-agnostic design corpus per §11.4.28/§11.4.51. Content is retained as historical design intent (§11.4.35/§11.4.124), not shipped software; the credentials/rootless-container/no-silent-removal mandates cited here ARE real and already govern this project's actual code independently of this document.

Table of Contents

1. Threat Model
2. Threat Vectors & Mitigations
3. Security Layers
4. Constitutional Security Mandates
5. Network Architecture
6. Incident Response

---

1. Threat Model

1.1 What We Protect

The SpecKit-Superpowers bridge system spans multiple trust domains:

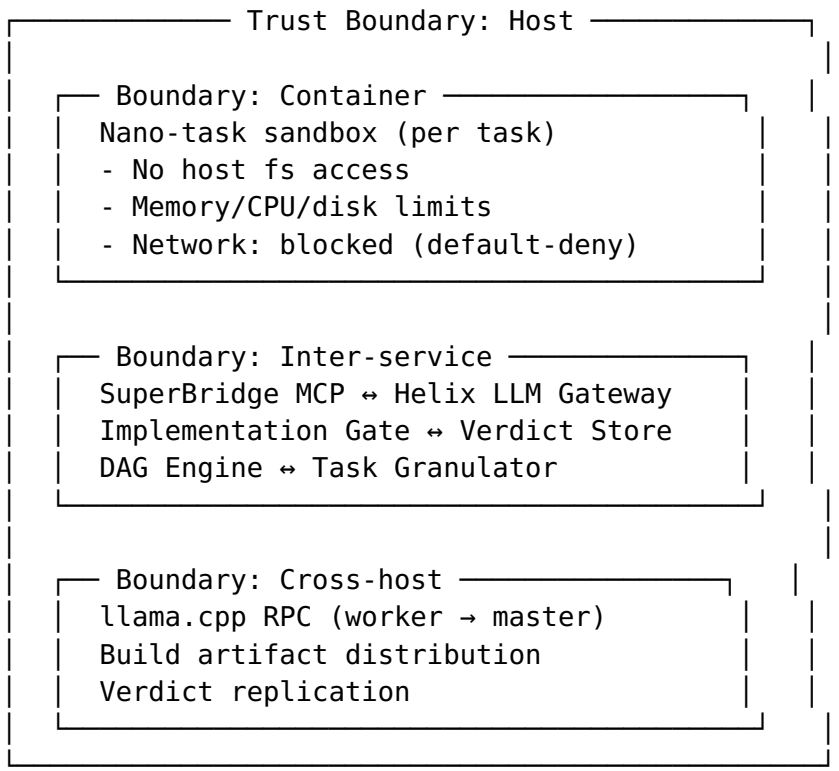
Asset	Sensitivity	Impact if Compromised
Nano-task source code	Moderate	Malicious code injection into the build pipeline
Task decomposition DAG	High	Reordering/dropping tasks → incorrect system behaviour
LLM prompt streams	High	Credential leakage through task descriptions (§11.4.10)
Implementation gate verdicts	Critical	Bluffed PASS → broken features shipped
Workable items database	High	Status manipulation, loss of evidence trail
llama.cpp RPC communication	Moderate	Intercepted/injected model inference
Distributed host GPU resources	High	VRAM exhaustion → denial of service
Container runtime	High	Host escape through container breakout

1.2 Threat Actors

Actor	Capability	Motivation
Malicious nano-task	Code execution in sandboxed container	Inject backdoor, exhaust resources, corrupt evidence
Compromised LLM endpoint	Intercepted inference requests	Model poisoning, credential exfiltration
Network adversary (MITM)	Packet capture on plain TCP RPC	Intercept llama.cpp inference, inject responses
Rogue subagent	Agent spawned with excessive permissions	Bypass gates, write to unauthorized paths
Insider (operator error)	Misconfiguration of secrets	Credential leakage, access token exposure

Actor	Capability	Motivation
Resource attacker	Crafted tasks causing infinite loops	GPU VRAM exhaustion, CPU pinning, DoS

1.3 Trust Boundaries



---

2. Threat Vectors & Mitigations

2.1 VRAM Exhaustion Attacks on GPU Nodes

Vector: A malicious or poorly-constructed nano-task submits an LLM inference with context size exceeding the GPU's VRAM capacity, causing OOM or forcing fallback to CPU (degrading all co-located inference workloads).

Mitigations:

1. **Per-task context budget enforcement.** The Helix LLM Gateway enforces a maximum context window per nano-task. Tasks exceeding the budget are rejected with a categorized error:

```go
// gateway/limits.go
type TaskContextBudget struct {
    MaxTokens    int `json:"max_tokens"`    // hard cap: 4096 for
        nano-tasks
    MaxKVSize    int `json:"max_kv_size"`   // estimated KV cache
        in MB
    MaxPromptLen int `json:"max_prompt_len"` // characters
}

func (g *Gateway) enforceBudget(task NanoTask) error {
    if task.EstimatedTokens > g.config.MaxTokens {
        return fmt.Errorf("REJECTED: task %s estimated %d tokens
        exceeds budget %d",
            task.ID, task.EstimatedTokens, g.config.MaxTokens)
    }
    return nil
}
```

1. **GPU health telemetry.** Continuous sampling of `nvidia-smi` / `rocm-smi` per GPU. VRAM usage tracked as deltas. Threshold alerts at 85% utilization trigger backpressure.

2. **Task isolation at GPU level.** Disjoint GPU assignment per task class (no single nano-task can consume all GPUs). Multi-host clusters distribute tasks across nodes.

3. **Recovery:** OOM-killed task is marked `crashed` per §11.4.147, respawned with halved budget.

2.2 Insecure RPC (llama.cpp Plain TCP)

Vector: llama.cpp's RPC protocol operates over plain TCP with no authentication, encryption, or integrity protection. An adversary on the same network can intercept inference requests (reading prompts), inject responses (model poisoning), or replay requests (resource exhaustion).

Proposed Mitigations:

1. **TLS wrapper (recommended).** Wrap the TCP RPC in mutual TLS (mTLS) using per-host certificates:

```
# TLS-wrapped llama.cpp RPC
llama-server \
  --host 0.0.0.0 --port 8443 \
  --ssl-cert /etc/llama/certs/server.crt \
  --ssl-key /etc/llama/certs/server.key \
```

```
  --ssl-ca /etc/llama/certs/ca.crt \
  --ssl-require-client-cert
```

1. **Isolated VLAN (fallback).** If TLS wrapping is not feasible (llama.cpp does not natively support it), isolate all RPC traffic to a dedicated VLAN with no external routing. The VLAN is provisioned via the Containers submodule (§11.4.76).

2. **Per-request HMAC.** Append an HMAC-SHA256 of the request body + a shared secret to each RPC message. The server validates before processing.

3. **Audit:** RPC traffic volume and timing is logged. Anomaly detection flags unusual patterns (burst of identical requests, requests from unknown IPs).

2.3 Verification-Loop Hangs (Maliciously Crafted Tasks)

Vector: A task is crafted to create an infinite TDD loop — a test that can never pass (asserting `1 === 2`) or a test whose RED phase depends on an unreachable condition. The implementation gate spins indefinitely, consuming resources and blocking the task queue.

Mitigations:

1. **Per-task timeout.** The implementation gate enforces a hard timeout (default: 300 seconds) per RED/GREEN phase. Exceeded timeout → mark `crashed` + systematic-debugging auto-trigger.

2. **Loop detection.** If the same item_id re-enters the RED→GREEN cycle more than N times (default: 5) without reaching GREEN, the implementation gate classifies it as a possible verification-loop attack and blocks further cycles:

```
const maxCycles = 5;
const cycleCount = await getCycleCount(itemId);
if (cycleCount > maxCycles) {
  return {
    verdict: 'BLOCKED',
    reason: `VERIFICATION_LOOP: ${itemId} has cycled RED→GREEN $
        {cycleCount} times without reaching GREEN. Possible
        malicious task or unfixable defect. Systematic debugging
        required.`
```

```
  };
}
```

1. **Resource budget per task.** CPU time, memory, and disk I/O are capped per nano-task via container limits.

2.4 State Drift Attacks

Vector: An attacker manipulates intermediate state (verdict JSONs, evidence files, DAG node status) to bypass gates. Example: overwriting a RED verdict's `exit_code` from 1 to 0 to satisfy GREEN-precondition-gate.

Mitigations:

1. **Content-addressed verdict store.** Every verdict file's path includes its SHA-256 hash. A modified file has a different hash and is no longer reachable by the gate:

```
qa-results/tdd_verdicts/
  RED_ATM-512_sha256=abc123def456.json
  GREEN_ATM-512_sha256=def789abc012.json
```

1. **Immutable append-only ledger.** All verdicts are also written to an append-only JSONL ledger (§11.4.116 sync channel). Any modification is detectable by comparing the ledger's hash-chain.

2. **Digital signatures.** Verdicts are signed with the conductor's key. The gate verifies the signature before accepting a verdict.

3. **Single-writer ownership.** Only the implementation gate process owns the verdict directory. Per §11.4.206, exactly ONE writer — every other consumer is read-only.

2.5 Credential Leakage Through Task Prompts

Vector: An operator's prompt or a task description contains an API key, token, or password. This text is serialized into a nano-task, transmitted over the network, logged, and potentially committed to the evidence store.

Mitigations:

1. **Pre-ingest credential scan (§11.4.10).** The task granulator scans every task description for credential patterns before creating nano-tasks. Matches are redacted:

```
const CREDENTIAL_PATTERNS = [
  /(?:api[_-]?key|apikey|access[_-]?token|secret|password|passwd)
        \s*[:=]\s*\S+/gi,
  /ghp_[a-zA-Z0-9]{36}/g,   // GitHub PAT
  /sk-[a-zA-Z0-9]{32,}/g,    // OpenAI key
  /xox[bpras]-[a-zA-Z0-9-]+/g // Slack token
];

function redactCredentials(text: string): string {
  let redacted = text;
  for (const pattern of CREDENTIAL_PATTERNS) {
    redacted = redacted.replace(pattern, '***REDACTED***');
  }
  return redacted;
}
```

1. **Credential-bearing tasks refused at creation.** If a task description contains a credential after redaction (the redacted text itself is a marker), the granulator refuses to create the task and surfaces an alert.

2. **Never-logged.** Logging infrastructure strips credential patterns before writing. Evidence files are scanned post-generation.

2.6 Task Injection (Malicious Code in Decomposed Tasks)

Vector: A compromised subagent or LLM injects malicious code into a nano-task's implementation. The TDD gate sees GREEN (the injected code also passes the test), but the shipped artifact contains a backdoor.

Mitigations:

1. **Independent code review (§11.4.142).** Every nano-task's implementation is reviewed by a structurally-separate agent BEFORE the GREEN verdict is accepted.

2. **Static analysis.** Each nano-task's implementation is scanned by the SonarQube scanner (§11.4.184) for common vulnerability patterns.

3. **Sandboxed execution.** Nano-task tests run in an isolated container with no network access and read-only host filesystem. Even if malicious code executes, its blast radius is bounded.

4. **Diff audit.** The commit diff for each nano-task is minimal (typically 10–50 lines). Unusually large diffs trigger manual review.

2.7 Model Poisoning (Tampered Model Weights)

Vector: The llama.cpp model weights are replaced with a tampered version that produces subtly incorrect or malicious outputs.

Mitigations:

1. **Model integrity verification.** Before loading, the Helix LLM Gateway verifies the model file's SHA-256 against a known-good hash stored in a git-tracked manifest:

```
func verifyModelIntegrity(modelPath, expectedHash string) error {
    f, err := os.Open(modelPath)
    if err != nil { return err }
    defer f.Close()

    h := sha256.New()
    if _, err := io.Copy(h, f); err != nil { return err }
    actual := hex.EncodeToString(h.Sum(nil))

    if actual != expectedHash {
        return fmt.Errorf("MODEL_INTEGRITY_FAIL: %s hash %s !=
        expected %s",
            modelPath, actual, expectedHash)
    }
    return nil
}
```

1. **Known-good hash registry.** `models/manifest.yaml` tracks every model variant, its expected hash, and the source URL.

2. **Read-only model storage.** The model file is mounted read-only into the inference container.

2.8 Cloud Provider MITM (Intercepted API Calls)

Vector: When falling back to a cloud LLM provider (after all local llama.cpp workers are at capacity), API calls traverse the public internet. An adversary at the cloud provider or on the network path intercepts or modifies API requests/responses.

Mitigations:

1. **Mandatory HTTPS.** All cloud provider API calls use TLS 1.3 with certificate pinning.

2. **Request signing.** Requests are signed with HMAC-SHA256 using a per-project secret. The provider-agnostic gateway layer adds the signature; the consumer verifies it.

3. **Provider fallback as last resort (§11.4.196).** Native local models are always preferred. Cloud providers are used only when ALL local workers are exhausted.

4. **Audit logging of all external calls.** Every cloud API call is logged with request hash, response hash, latency, and provider identity.

---

3. Security Layers

3.1 Constitution-Enforced Credential Handling (§11.4.10)

- All credentials live in `.env` files (git-ignored).
- Templates with placeholder values are committed as `.env.example`.
- Pre-commit hook scans staged files for credential patterns.
- Tests load credentials at runtime, never hardcoded.
- Credential-bearing frames in recordings are redacted.
- Per-service credential separation (`.netflix.env`, `.disney.env`).

3.2 TLS / mTLS for Inter-Service Communication

Service Pair	Protocol	Auth
SuperBridge MCP ↔ Helix LLM Gateway	mTLS (TLS 1.3)	Client cert
Implementation Gate ↔ Verdict Store	Local Unix socket	Filesystem permissions
DAG Engine ↔ Task Granulator	mTLS (TLS 1.3)	Client cert
llama.cpp RPC (worker → master)	TLS wrapper or isolated VLAN	Cert or network isolation
Cloud provider API	HTTPS (TLS 1.3)	API key + request signing

Certificate management via `constitution/scripts/certs/` (shared tooling, inherited by reference per §11.4.177).

3.3 Sandboxed Nano-Task Execution

Every nano-task executes in an isolated container provisioned via the `vasic-digital/containers` submodule (§11.4.76). Container profile:

```yaml
# container profile: nano-task-sandbox
services:
  nano-task:
    image: "nano-task-runner:latest"
    read_only: true
    tmpfs:
      - /tmp:size=128M
    mem_limit: "256m"
    cpus: "0.5"
    pids_limit: 50
    network_mode: "none"
    security_opt:
      - no-new-privileges:true
    cap_drop:
      - ALL
    cap_add: []  # zero capabilities
    user: "1000:1000"  # non-root
    volumes:
      - type: bind
        source: ./task_input
        target: /task/input:ro
      - type: bind
        source: ./task_output
        target: /task/output
```

Key invariants:

- No network access (network_mode: none) — nano-tasks cannot exfiltrate data.
- Read-only root filesystem except `/tmp`.
- Non-root user.
- Dropped ALL Linux capabilities.
- Per-task memory/time/PID limits.
- Rootless Podman (§11.4.161) — no rootful Docker.

3.4 Input Validation at Every Layer

Layer	Validation
Task Granulator (granulator.ts)	Schema-validate task spec against JSON Schema
DAG Engine (dag.go)	Validate graph is acyclic, all dependencies resolve

Layer	Validation
Implementation Gate	Validate test is not a tautology, verify RED/GREEN chain
Helix LLM Gateway	Enforce context budget, sanitize prompts, verify model integrity
Verdict Store	Validate verdict JSON schema, verify signature
External Tracker Sync	Validate against tracker field map, idempotency-key

3.5 Audit Logging with Immutability

All security-relevant events are written to an append-only, integrity-protected audit log:

```
{
  "event": "nano_task_execution",
  "item_id": "ATM-512",
  "phase": "GREEN",
  "container_id": "abc123",
  "exit_code": 0,
  "resource_usage": { "cpu_sec": 1.2, "mem_mb": 45, "disk_kb":
        120 },
  "sandbox_escapes_detected": 0,
  "timestamp": "2026-07-24T12:05:00Z",
  "signature": "sha256:..."
}
```

The audit log is:

- Append-only (no deletion, no modification).
- Hash-chained (each entry includes the hash of the previous entry).
- Replicated to a separate host (tamper-evident).
- Monitored for anomalies (unusual exit codes, resource spikes, repeated failures).

---

4. Constitutional Security Mandates

4.1 Credential Handling (§11.4.10)

> "Credentials or any secret and sensitive data MUST NOT leak!"

Every credential path and pattern is documented in the consuming project's declared canonical credential registry (the single source of truth — its concrete path is project-specific DATA per §11.4.35, never hardcoded in this project-agnostic design corpus). The pre-commit hook, the credential leak scan, and the task-granulator prompt scanner form a defense-in-depth barrier.

4.2 Rootless Containers (§11.4.161)

> "Every project MUST use Podman in rootless mode for ALL containerized workloads."

All nano-task sandboxes, build containers, and service containers run rootless. No `sudo`, no Docker daemon, no root-equivalent capabilities. Container-to-root privilege escalation is structurally prevented, not merely gated.

4.3 Credential Single-Source-of-Truth (§11.4.10, project-agnostic)

> **Genericised 2026-09-22 (HXC-159 Phase B3).** This section previously read "(ATMOSphere project mandate)" and hardcoded the path `docs/accesses/accounts.md` — a different project's own convention that does not belong in this project-agnostic design corpus, per §11.4.28/§11.4.51 decoupling (a reusable submodule-adjacent design document must never hardcode another project's name or paths). Corrected below to a project-neutral statement of the same §11.4.10 credential single-source-of-truth mandate; the consuming project supplies its own concrete registry path as DATA per §11.4.35.

The consuming project's declared canonical credential registry file is the single source of truth for credentials. Any new credential MUST be added there. Any credential change MUST update it. Credentials MUST NOT leak through any other file.

4.4 No-Silent-Removal (§11.4.122)

Security components (credential scanners, sandbox profiles, TLS certificates, audit logging) are end-user capabilities. They MUST NOT be removed without explicit operator confirmation.

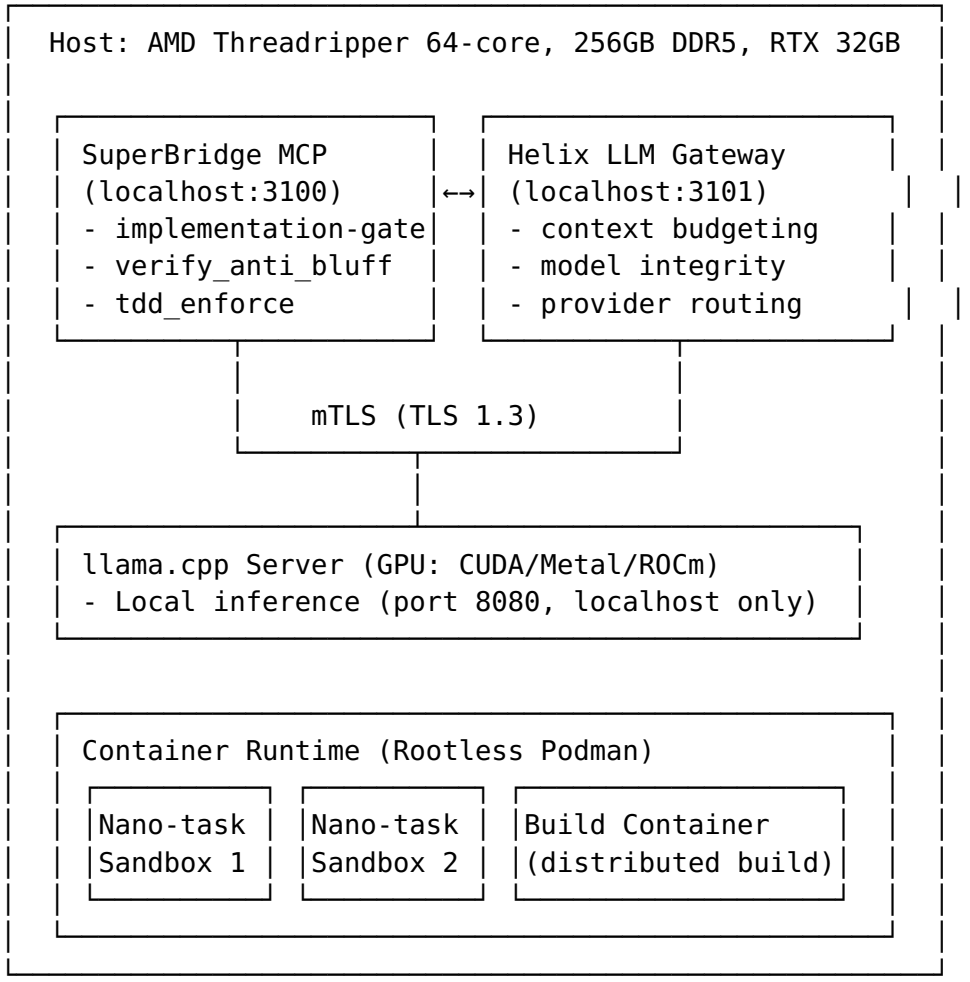
4.5 Target-System + Hardware Safety (§11.4.133)

Every security change MUST be safe for BOTH the target system (MUST NOT brick, boot-loop, or corrupt data) AND the hardware (MUST NOT exceed electrical/thermal/voltage limits).

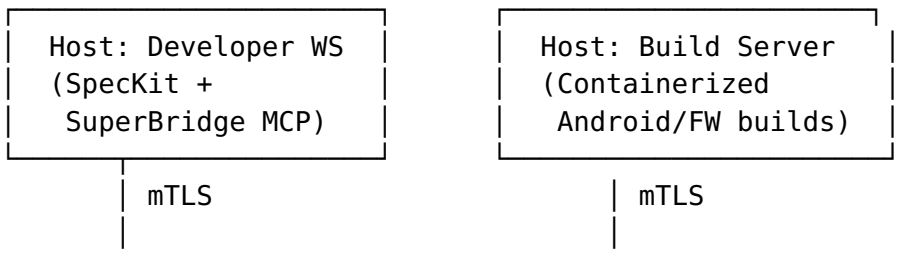
---

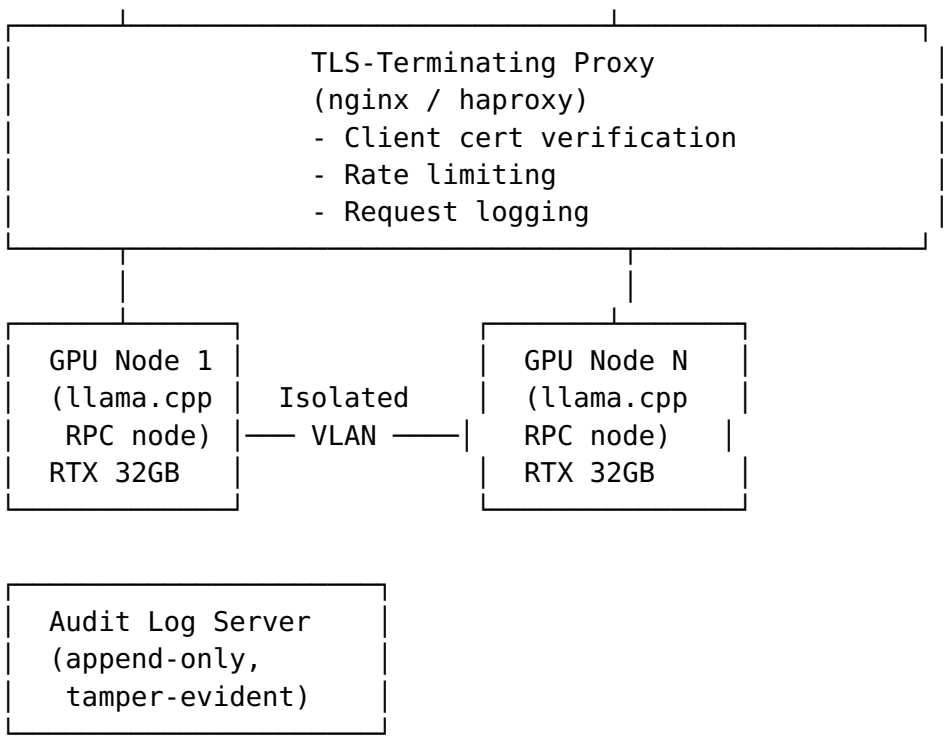
5. Network Architecture

5.1 Single-Host Deployment (Development)



5.2 Multi-Host Deployment (Production)





5.3 Security Zones

Zone	Network Access	Services
Green (Internal)	None	Nano-task sandboxes
Yellow (Service Mesh)	mTLS-only, localhost	SuperBridge MCP, Gateway, Verdict Store
Orange (GPU Compute)	Isolated VLAN	llama.cpp RPC nodes
Red (External)	HTTPS-only, rate-limited	Cloud LLM providers
Blue (Audit)	One-way push	Audit log replication target

---

6. Incident Response

6.1 Detection

Security events are automatically detected by:

1. **Implementation gate anomalies:** Repeated RED→GREEN loops on the same nano-task (loop detection), unusual exit codes, container escape attempts.

2. **Audit log monitoring:** Anomalous resource usage, unexpected container exit codes, modified verdict files (hash mismatch).
3. **Credential scan alerts:** Pre-commit hook rejects, task-granulator redaction events.
4. **Network traffic anomalies:** Unexpected RPC sources, burst traffic patterns, TLS certificate mismatches.

6.2 Response Procedures

Level 1 — Credential Leak Detected:

1. Pre-commit hook or credential scan triggers.
2. The offending text is redacted or the commit is blocked.
3. The operator is notified of the exact file + line + pattern detected.
4. If the credential was pushed, rotate the credential at the provider immediately.
5. Audit git history for the leaked value's presence.
6. Document in `docs/audit/credential_leak_<date>.md`.

Level 2 — Suspicious Nano-Task Detected:

1. Implementation gate's loop detection fires.
2. The nano-task is marked `BLOCKED` (not crashed — different recovery path).
3. Systematic debugging is auto-activated per §11.4.102(D).
4. The task's source, the test, and the DAG path are manually reviewed.
5. If confirmed malicious: isolate the subagent that created it, revoke its credentials, audit all its other tasks.
6. If false positive: adjust loop detection threshold, record the finding.

Level 3 — Container Escape or Host Compromise:

1. Isolate the affected host from the network immediately.
2. Stop all running containers on that host.
3. Preserve forensic evidence: container logs, audit logs, filesystem snapshots.
4. Investigate the escape vector (kernel vulnerability? misconfiguration?).
5. Patch and redeploy.
6. Full post-incident audit per §11.4.138 (what assertion should have caught this?).

Level 4 — Model Poisoning or Provider Compromise:

1. Model integrity check fires (hash mismatch).

2. The compromised model is quarantined (moved, not deleted — preserve for forensics).
3. All recent inference results using that model are flagged for re-verification.
4. The known-good model is restored from backup (hash-verified).
5. If a cloud provider is the source: revoke API keys, rotate, investigate with provider.

6.3 Post-Incident Obligations

Per §11.4.138, every security incident MUST produce:

1. A systematic-debugging root-cause analysis.
2. A bluff-audit identifying the exact assertion/check that should have caught it.
3. A permanent regression guard registered in the same commit as the fix.
4. The bluff-audit committed under `docs/research/security/<incident>_bluff_audit/`.

End of Security Architecture & Threat Model.